

Guia de Estudos: Alta Computação (Tarefas 9 a 21)

Este documento contém um resumo detalhado e organizado por tópicos das tarefas 9 a 21, focando nos conceitos estudados, nas diretivas e rotinas (OpenMP e MPI) e na explicação do que foi implementado e analisado em código. Este material foi preparado para auxiliar na revisão para a prova oral.

Tópico 1: Sincronização e Concorrência em Memória Compartilhada (OpenMP)

Neste tópico, o foco principal é lidar com o acesso concorrente a recursos compartilhados por múltiplas *threads*, garantindo a corretude dos dados sem sacrificar excessivamente o desempenho através da exclusão mútua.

Tarefa 9: Sincronização em Lista Encadeada

Conceitos Estudados: Nesta tarefa, foi simulada a inserção concorrente em listas encadeadas usando diferentes mecanismos de sincronização do OpenMP. O principal desafio é a **condição de corrida**: se duas *threads* tentam inserir um elemento na mesma lista simultaneamente, os ponteiros da estrutura de dados se sobrepõem e corrompem a lista.

Cláusulas e Mecanismos (OpenMP):

- `#pragma omp critical`: Cria uma região crítica global. Apenas uma *thread* por vez em todo o programa pode executá-la. Na versão “sem nome”, todas as inserções são serializadas, o que cria um gargalo imenso, mesmo que as *threads* queiram acessar listas independentes.
- `#pragma omp critical(nome)`: Permite criar seções críticas independentes baseadas no nome (ex: `critical(lista1)`). *Threads* que tentam inserir em listas diferentes não bloqueiam umas às outras, paralelizando os acessos ortogonais. Contudo, não resolve a contenção quando as *threads* disputam efetivamente a mesma lista simultaneamente.
- `omp_set_lock` e `omp_unset_lock`: Rotinas de mais baixo nível da API do OpenMP que funcionam como variáveis *mutexes*. Permitem trancar uma estrutura de dados de forma explícita e modular (lock por lista).

O que foi feito no código e nas avaliações: Foram implementadas as três abordagens inserindo valores de forma aleatória em duas listas (`target_list`) calculadas com um `rand_r` seguro por *thread*. Executando 50 milhões de operações, constatou-se que o `#pragma omp critical` comum engessava a execução serializando-a no funil. As abordagens `critical(nome)` e o lock modularizado (`omp_set_lock`) mitigaram as travas desnecessárias, obtendo desempenho até 60% superior de balanceamento.

Tarefa 10: Estimativa Estocástica de Pi (Monte Carlo)

Conceitos Estudados: Estimativa do número Pi sorteando iterativamente pontos aleatórios (x, y) e verificando se sua distância em relação à origem garante que eles caíram dentro de um círculo unitário. Apesar de tratar-se de loops perfeitamente paralelos, todas as threads demandam incrementar o **mesmo** contador global de “pontos acertos”.

Cláusulas e Mecanismos (OpenMP):

- `#pragma omp atomic`: Diretiva suportada via *hardware* para operações pontuais rápidas (como incrementos `++` ou somas `+=`) em uma única variável na memória compartilhada, substituindo a pesada carga burocrática de bloquear e liberar mutexes presente no *critical*.
- **Variáveis Privadas**: Para evitar bater no endereço de memória global 100 milhões de vezes com `atomic` ou `critical`, pode-se criar um *contador local exclusivo por thread*. Ao fim do ciclo de iterações maciças, cada thread descarrega sua soma efetuando um único acúmulo no contador compartilhado.
- `#pragma omp for reduction(+:variavel)`: Abordagem nativa suprema do OpenMP que encobre o detalhamento manual das variáveis privadas. Ela deduz locais para as threads do laço `for` acumular e automaticamente soma todas na variável global ao encerrar o escopo num fluxo protegido e seguro.

O que foi feito no código e nas avaliações: Testou-se versões baseadas no sorteio seguro para threads usando a `rand_r` e uma *seed* diferente por *rank*. O código que chamava `#pragma omp critical` e `#pragma omp atomic` em cada acerto de iteração pagou imensos overheads de tempo (acima de 2.0s e 0.60s respectivamente). Em contraponto, as soluções que separaram lógicas matemáticas (variáveis privadas com um sincronismo residual ao fim ou uso da elegante tag de `reduction(+ : pontos_acertos)`) estraçalharam as métricas baixando o tráfego e entregando a solução na média dos 0.03 segundos.

Tópico 2: Escalonamento, Desempenho e Afinidade (OpenMP)

O OpenMP oferece amplo gerenciamento sobre como as iterações de laços (loops) gigantescos são repartidas entre o número de *threads* geradas e as define junto as ramificações e cachês da arquitetura do processador.

Tarefa 11: Navier-Stokes e Viscosidade (collapse)

Conceitos Estudados: Simulação de difusão de velocidade num campo contínuo de fluidos 2D. O cálculo de estêncil iterando x e y exige cálculos algébricos perante nós espaciais baseados em diferenças finitas ($\Delta t, \Delta x, \Delta y$).

Cláusulas e Mecanismos (OpenMP):

- `#pragma omp parallel for schedule(static)`: Divide as iterações do laço uniformemente em grandes fatias estáticas entregues antecipadamente ao pool das threads de trabalho.
- `collapse(N)`: Cláusula que transforma N laços aninhados independentes (como o clássico `for(i)` seguido do `for(j)` na navegação da matriz 2D) num único e extenso hiperlaço linear subjacente ao *schedule*.

O que foi feito no código e nas avaliações: Realizou-se perturbações unitárias em arrays de grade usando `schedule(static) collapse(2)`. A cláusula estendeu eficientemente as bordas de processamento para além de fatiar apenas a dimensão primária da matriz (ex: as linhas). Analisou-se que a invocação de malhas em `collapse` impõe considerável custo administrativo inicial da biblioteca ao remapear as coordenadas do estêncil, de forma que execuções simples utilizando poucas *threads* mostraram speedups negativos. Só há um pico positivo real quando a máquina destrava quatro ou mais núcleos verdadeiros e amortece o peso num processamento simultâneo de escala maciça de iterações achatadas.

Tarefa 12: Escalabilidade no Supercomputador (Strong / Weak Scaling)

Conceitos Estudados: Bateria de validações de desempenho num cluster multicore (NPAD particionado intel-128) utilizando os agendadores algorítmicos fornecidos nativamente no OpenMP a fim de observar a Escalabilidade Forte (*Strong Scaling* - malha e limites rigorosamente constantes enquanto provisiona mais CPUs) e a Escalabilidade Fraca (*Weak Scaling* - eleva a área computável na mesma proporção do aumento de recursos adquiridos).

Cláusulas e Mecanismos de Agendamento (OpenMP):

- `schedule(dynamic, chunk)`: Repasse porcionado “sob demanda”. Se uma thread resolve depressa (malha desequilibrada onde blocos vazios pulam iterações inteiras), ela pede novo chunk ao coordenador. Encarece o laço pelo frequente overhead nas devoluções de chamadas de bloqueio à biblioteca OpenMP.
- `schedule(guided)`: Regime híbrido sofisticado. Larga o processo doando nacos gigantes de chunks para eliminar fatias de forma veloz poupando a biblioteca mas à medida que o limite de *iterations* se encerra, desce drasticamente o tamanho das requisições forçando um ajuste fino nas últimas threads desocupadas da fila, suavizando o balanceamento da carga remanescente.

O que foi feito no código e nas avaliações: Rodou-se simulações padronizadas do método Navier-Stokes variando os agendamentos (`static`, `dynamic`, `guided`). Demonstrou-se que perante iterações simétricas densas, os pacotes estáticos com *collapse* varrem o processamento com tempos brutos estritamente inferiores (os mais curtos no cronômetro do *Supercomputador*). Não obstante, no cômputo matemático isolado e rigoroso das frações de *Speedup Relativo* de 1 Core pra 16 Cores o agrupamento de `dynamic` e `guided` bateu escalas de performance incríveis alcançando $12x$, compro-

vando possuírem curvas de aceleração algorítmicas muito mais elásticas com a provisão superabundante da infraestrutura.

Tarefa 13: Afinidade de Threads (Thread Affinity)

Conceitos Estudados: Sistemas Operacionais, na falta de diretrizes, migram os *worker threads* arbitrariamente perante seus núcleos lógicos conforme sua livre concorrência da tabela de agendamento (Context Switch do Linux). A afinidade trava as instruções OpenMP de maneira persistente aos núcleos físicos, aos soquetes base ou as subdivisões SMT (Hyper-Threading).

Variáveis de Ambiente Exclusivas:

- **OMP_PLACES:** Limita a camada sistêmica para cores (restrito aos núcleos físicos), sockets (amarra nos chiplets independentes da CPU) ou threads (amarra à topologia das hardware-threads lógicas inter-chip).
- **OMP_PROC_BIND:**
 - **close:** Aloca adjacências. Preenche blocos sequenciais encavando instâncias nas caches de nível 1 e nível 2 compartilhadas (benéfico caso os fluxos exijam tráfego pesado e recíproco nas mesmas tabelas locais, minimizando latências).
 - **spread:** Preenchimento distributivo esparso. Ampla distribuição da escala para balancear cargas térmicas pelo silício completo e forçar exploração agressiva de toda a largura de banda separada (cache L3) nos conjuntos sem afetar ou afogar o controlador unificado de memória.

O que foi feito nas avaliações: Bateria de submissões comprovando categoricamente os benefícios da fixação da restrição estrita das afinidades em oposição às execuções nativas flutuantes. Os apontamentos firmados restritamente nas hardware-threads amarrados por proximidade (threads / close) garantiram speedups contundentemente superiores isolando os ruídos paralelos na máquina do laboratório.

Tópico 3: Troca de Mensagens e Memória Distribuída (MPI)

Saindo do paralelismo compartilhado dentro de uma única chapa de silício, a biblioteca *Message Passing Interface* (MPI) orquestra trocas via barramento da rede externa perante clusters monstruosos compostos por milhares de computadores independentes com memórias fechadas.

Tarefa 14: Ping-Pong com MPI (Latência e Largura de Banda)

Conceitos Estudados: Identificar o comportamento oscilatório do atraso inicial para disparar conexões na rede inter-nó contra as altas taxas limiares no descarregamento linear em fluxos densos.

Rotinas Clássicas de Comunicação Ponto-a-Ponto (MPI):

- **MPI_Send (Standard):** Modelo tradicional que infere pragmaticamente qual sistema operacional, buffer e sub-rotinas são mais rápidos (se as remessas são minúsculas ele usa buffers do O.S, se amplas engata um protocolo imediato direto via rede).
- **MPI_Ssend (Synchronous):** Comunicação restritamente bloqueante que garante que um laço for no programa só recarregará na memória se do outro lado o remetente notificar explicitamente via rede que engoliu e recebeu no MPI_Recv. Indispensável onde desvios da geometria causariam inconsistência de pacotes de dados globais.
- **MPI_Bsend (Buffered):** Rotina desvinculada rápida baseada no armazenamento customizado explicitamente em software pelo desenvolvedor via invocação da diretiva MPI_Buffer_attach. Extrapola o acúmulo da memória do O.S para mitigar desyncs entre remetentes agressivos.
- **MPI_Rsend (Ready):** Somente liberado sob circunstâncias estritas de *handshake* onde é inabalável que a recepção no nodo receptor abriu sua flag com antecedência, minimizando tratativas desnecessárias entre os roteadores da arquitetura interna.

O que foi feito nas avaliações: Programa recursivo trocando malhas de matriz de 8 bytes a 1 MiB. Mensurou-se que comunicações leves submetem a malha majoritariamente ao imposto fixo imutável do roteador na **latência**. Ao escalar pacotes grandes acima da centena de kilobytes a arquitetura infla as curvas linearmente batendo no estrangulamento da placa física perante a máxima requisição de sua **largura de banda**. O MPI_Send manteve excelência nativa provando a alta e inigualável capacidade dos compiladores na abstração ótima generalista das decisões operacionais.

Tarefa 15: Difusão de Calor 2D e Sub-rotinas (Halo Cells e Overlap)

Conceitos Estudados: Surgimento de simulações com particionamento sub-estrutural espacial da matriz. Se processador A detém fatias da área norte de uma malha 2D, seu escopo exigirá os apontamentos matemáticos vizinhos situados internamente no processador B (ao sul) calculados via estêncil sobre fronteiras compartilhadas.

Rotinas de Comunicação Assíncrona e Escondendo Latência (Overlapping):

- As chamadas clássicas MPI_Send/Recv perante envio simultâneo de milhões de fronteiras engessarão redes causando interrupções mortais (*Deadlocks* - “eu aguardo sua frente e você aguarda minhas costas e nós duas congelamos”).
- A introdução explícita em arquiteturas assíncronas exige instanciar aberturas passivas subjacentes ao hardware pelo MPI_Isend e MPI_Irecv. Elas apenas indicam e liberam as requisições em background pelas placas Infiniband despachando o controle da CPU local imediatamente.
- Exige ao final fechar os gargalos validando as instâncias engavetadas da rede utilizando portas com a função de encerramento coletivo MPI_Waitall.
- O pico absoluto algorítmico do paralelismo se concretiza sobrepondo (*Overlap*) as subrotinas. Enquanto a placa de rede mastiga latências copiando dados nas fronteiras via requests do hardware, os núcleos do C++ processam isoladamente cálculos do centro limpo da matriz no núcleo silício. Um ganho abissal sem esperas mortas.

O que foi feito nas avaliações: Três estratégias num problema denso (matriz 4000x4000) submetidas num agrupamento de 8 nós isolados no laboratório do NPAD de modo que as transições envolveram o protocolo da rede real Eager-Rendezvous. Validação categórica que o tráfego estritamente escondido sob o Overlapping do hardware absorveu o processamento central tornando a simulação uma fração inigualavelmente super veloz em relação as metodologias blocantes tradicionais.

Tarefa 16: Escalonador Dinâmico e Desbalanceamento Físico (Líder-Trabalhador)

Conceitos Estudados: Abordagem pragmática do problema onde partições divididas igualmente não possuem processamentos equivalentes (determinar um número primo astronômico custa ciclos severamente mais onerosos do que constatar primos de dezenas singulares em laços curtos). Divisão estática fatalmente condena nós a desativarem-se cedo causando engarrafamentos ociosos num supercomputador caríssimo enquanto outras máquinas carregam simulações gigantes na costas sem escalabilidade da rede.

Mecanismos Distribuídos e Controle Mestre-Escravo:

- Destitui-se o processamento num **Líder** despachante burocrata e as massas em **Trabalhadores** (Workers) isolados. O coordenador expede tarefas (WORKTAG) de pacotes intervalares, recebe por meio de canais genéricos de finalização da placa usando identificadores coringas (MPI_ANY_SOURCE) atestando a ociosidade do destinatário que recém-acabou e devolvendo imediatamente novos envios fragmentários dinâmicos.
- Ao abater as listas, expede ordens de pílula-venenosa (DIETAG / Poison-Pill) aos pares requisitantes informando a extinção e repouso uníssono das unidades finalizando o MPI perfeitamente em simetria de deadlock-free.
- *Granularidade:* Balancear se os pacotes (chunks) são massivos e ineficientes criando resíduos pendentes no final ou esfarelados em poeiras sobrecarregando a latência do roteador.

O que foi feito nas avaliações: Varredura de caça à milhões de primos engajando instâncias de 1x, 3x e 7x trabalhadores com pacotes imensos contra pacotes microscópicos na variação da contagem bruta. Averiguou-se as métricas estatísticas da Eficiência (E) despencarem de 98% no nodo unitário para ~72% com agrupamento das centenas de Cores evidenciando gargalo administrativo contínuo do Nó-0 principal, mas comprovando a fluidez máxima estabilizando picos num balanço em blocos de 100K garantindo desfechos ideais mitigados à estagnações arbitrárias de finalizações pendentes.

Tarefas 17 e 18: Multiplicação Matriz-Vetor (Distribuições Híbridas)

Conceitos Estudados: Análise implacável nas complexidades limitantes do teorema de redes (Memory-bounds e Communication-bounds) em arquiteturas interligadas ao tentar isolar e acelerar cálculos triviais ($O(N^2)$) despendendo esforços desnecessários

na rede. Impactos abissais de abordagens lógicas puras em vetores colunares e linhas frente a disposições matriciais contíguas físicas da linguagem C++.

Cláusulas Avançadas de Empacotamento Coletivo do (MPI):

- Invocação de divisões massivas entre todos via descarregamentos simétricos do MPI_Bcast (cópias absolutas do vetor).
- Fatiamentos de envios particionados com repasse em esteira de frações intercortadas nas malhas do nó coordenador usando as chaves de distribuição do MPI_Scatter e suas respectivas agregações e unificações ordenadas das malhas recebidas utilizando reincorporações por aglomeração no MPI_Gather.
- Tipos de Memórias Saltadas: Como uma sub-divisão nas linhas gera envio contíguo dos bytes em fileiras, no cenário contrário a geometria das simulações num fatiamento vertical transcorre através de pulos espaciais (strides). MPI exige acúmulos lógicos virtuais englobando alocações relativas MPI_Type_vector mitigadas nas chancelas subjacentes perante realinhamentos literais da placa da extensão de blocos através da correção estrita MPI_Type_create_resized.

O que foi feito nas avaliações e a Resolução dos “Mitos”: Um teste de proporções épicas que subiu alocações da matriz aos domínios além dos 10 Gigabytes da placa-mãe. Encarou-se a futilidade de clusters em algoritmos matriciais minúsculos. Constatou-se severamente que a divisão estrita nativa tradicional via Row-major (T-17) executou as quebras, o tráfego inter-rede InfiniBand, e as colagens com altíssima otimização em 0.00 falhas, esmagando e estraçalhando temporalmente a variação de envios de pacotes customizados da Column-major (T-18). Esta última arrasta a latência obrigando a confecção exaustiva da criação inter-buffer nos roteadores baseados em derivações e ainda despacha todo acúmulo reverso de volta nos nós exigindo imersas complexidades algébricas em redes pela invocação ineficaz das devoluções do sub-pacote global com a rotina pesada iterando por cima dos barramentos em chamadas de agregações como MPI_Reduce c/ MPI_SUM.

Tópico 4: Offloading para GPU (OpenMP Target)

Uso pragmático e maciço da linguagem OpenMP para expor diretrizes nativas das abstrações de aceleração e forçar os descarregamentos (offload) transfronteiriços de loops matriciais de paralelismo numérico agressivo aos milhares de Streaming-Multi-processors de placas aceleradoras super escaláveis (GPGPU V100).

Tarefa 19: Vector Addition (Gargalos de Baixa Intensidade Aritmética)

Conceitos Estudados: O processamento GPU envolve a remessa física explícita entre a memória RAM centralizada da Placa-Mãe e as matrizes internas exclusivas alocadas isoladamente sob o controle gráfico da arquitetura na Memória de Vídeo (VRAM). Barramentos físicos do hardware (PCI-Express) figuram custos latentes substanciais que inibem offloads perante matrizes irrisórias.

Cláusulas e Mecanismos (OpenMP Target):

- A flagura mandatória engajando a execução à placa `#pragma omp target`.
- Englobamento interno na hierarquia algorítmica forçando paralelizações e alinhamentos perante blocos lógicos da interface em concorrência pelo escopo numérico do laço via diretriz simples nativa `#pragma omp loop`.

O que foi feito nas avaliações e A Falácia do Hardware: Somas aritméticas primitivas ($c = a + b$) processadas sobre dez milhões de coordenadas e flutuantes. Expondo tempos inauditos onde a CPU de 18 Cores trucidou o Hardware V100 milionário fechando os pacotes na singular marca esmagadora de 0.011 segundos frente ao lentíssimo desespero nos 0.35 segundos do acelerador. Comprovando que arquiteturas GPGPU não aceleram cálculos com baixíssimo teto das equações na *Intensidade Aritmética*. Todo o custo colossal de arrasto foi a latência física isolada dos pinos banhados à ouro do conector PCIE escoando milibytes infinitos enquanto o laço era exaurido nos primeiros nanosegundos do multiprocessador matricial.

Tarefa 20 e 21: Otimização Extrema Heat Transfer nas VRAM

Conceitos Estudados: Mapeamento agressivo nos tempos estáticos da abstração das camadas de arquiteturas perante simulações densas (Diferenças Finitas da Dispersão Térmica de Fluidos iteradas em matrizes espaciais) encapsulando blocos de modo a banir estritamente todo descarregamento recursivo nos limites de comunicação por ponteiros, mantendo todos os volumes computacionais fixados na NVIDIA e mitigando os barramentos de interligações.

Cláusulas Exclusivas Avançadas de Escopo e GPU Mapping:

- Uma chamada purista de GPU num laço principal em `#pragma omp target` num ambiente de *Timesteps* arrastaria o código e obrigaria envios PCIE host-device perante **CADA** quadro da matriz, aniquilando a funcionalidade e trucidando sua escalabilidade frente a soluções CPUs-baseline.
- Invocações macro estruturais da diretiva de ciclo de vida com fixação mandatória através da declaração de bolhas do tipo `#pragma omp target data map(to: matriz)` mitigando o custo numérico. Instancia todo volume com os arranjos dimensionais no início. Roda todos os milhares de *Timesteps* sem comunicar externamente nenhuma vez à Placa Mãe e encerra desvios do cálculo exportando do hardware isoladamente com sub-declarações modulares ou de requisição forçadas através da tag `global update from()`.
- Abstrações explícitas ou classes orientadas sem demarcações claras são manipuláveis estritamente no tempo físico usando diretrizes fracionadas desestruturantes alocáveis perante `#pragma omp target enter data / exit data`.
- Mapeamentos espaciais das *threads* hierárquicas da malha de silício CUDA na invocação do pragmatismo puro `#pragma omp target teams distribute parallel for collapse(...)` aglomerando blocos em grupos de malha distribuíveis uniformemente.

mente perante os SMs, elevando a densidade dos laços aninhados ao poder gráfico nativo.

O que foi feito nas avaliações: A revolução das placas dedicadas para problemas de altíssima relevância. Malhas massivas gigantescas de simulação climática simuladas sobre blocos 8000 x 8000 nos vetores. As chamadas em matrizes CPU processadas a duras penas esmagando trinta extenuantes segundos ($\sim 30.0s$) caíram e despencaram agressivamente engolindo e cuspiendo resultados super massivos em exímios frações inaudíveis e insondáveis perante a estrita marca abissal dos **0.51 segundos** utilizando isoladamente a arquitetura desvinculada NVIDIA perfeitamente balanceada e livre da escravização das latências, elevando processamentos em esmagadores incrementos insubstituíveis e evidenciando a coroa e a supremacia das GPUs perante ambientes pesados com laços aninhados massivos baseados em interações iterativas.